

strings in our app:

```
<string name="app_name">Bio Builder</string>
<string name="profile_pic">Profile Picture</string>
<string name="edit_profile">Edit profile</string>
<string name="update_profile_picture">Update picture</string>
<string name="text_template">%1$s: %2$s</string>
<string name="name_label">Name</string>
<string name="about_label">About</string>
<string name="phone_label">Phone</string>
<string name="email_label">Email</string>
<string name="dob_label">Date of Birth</string>
<string name="save_label">Save</string>
<string name="default_name">Default name</string>
<string name="default_about">I am a software developer.\nEdit this bio by clicking on the Edit button at the top</string>
<string name="default_phone">(123) 456-7890</string>
<string name="default_email">default.name@example.com</string>
<string name="editable_app_title">Edit Bio</string>

<string name="choose_image">Choose an image</string>
```

Bio-page: *EditButton*

Create a composable function called *EditButton*. Composable functions can be created by adding the *@Composable* prefix.

```
@Composable
fun EditButton(
    modifier: Modifier = Modifier,
    onEdit: () -> Unit
) {
    IconButton( modifier = modifier, onClick = onEdit) {
        Icon(
            imageVector = Icons.Default.Edit,
            contentDescription = stringResource(id =
R.string.edit_profile)
        )
    }
}
```

Inside the *EditButton* composable, we can add an *IconButton*. We use the *Modifier* object to apply parameters like padding, size, etc. *onEdit* lambda is called when the button is clicked. We call an *Icon* composable with *Icons.Default.Edit* as the *imageVector* inside the content lambda.

Now create a state variable for storing the value of *editable* inside the *setContent* composable:

```
var editable by rememberSaveable {
    mutableStateOf(false)
```

```
}
Create a lambda for enabling the editing:
```

```
val onClickEdit = {
    editable = true
}
```

Create another lambda for cancelling the editing:

```
val cancelEdit = {
    editable = false
}
```

Now, we can preview our *EditButton* using the *@Preview* annotation. Create another composable function named *PreviewEditButton*. Inside the *PreviewEditButton*, call *EditButton* composable. Pass an empty lambda as *onEdit* parameter. If we build the application now, you can see our *Edit* button.

```
@Preview
@Composable
fun PreviewEditButton() {
    EditButton {
    }
}
```

Profile picture

We can now create a composable for our profile picture.

Create a composable function called *ProfileImage*. This function will show an *Edit* button in edit mode. On clicking the *Edit* button, it will show an option to pick a profile picture. This function will have three other parameters along with a modifier parameter -- a Boolean value to determine the mode, an ID of the profile picture, and a lambda for changing the image upon clicking the *Edit* button.

```
@Composable
fun ProfileImage(
    modifier: Modifier = Modifier,
    editable: Boolean,
    picId: Int,
    onChangePic: () -> Unit
) {
    Box(
        modifier = modifier,
        contentAlignment = Alignment.BottomCenter
    ) {
        Image(
            modifier = Modifier.size(128.dp),
            painter = painterResource(id = picId),
            contentDescription = stringResource(id =
R.string.profile_pic)
        )
        AnimatedVisibility(visible = editable) {
```